

Lecture Indexer: A Multi-Cloud Service for Timestamped Keyword Search of Recorded Lectures

Theodore Podewil

Dept. of Computer Science
Western Michigan University
Kalamazoo, MI, USA

Joshua Jubenville

Dept. of Computer Science
Western Michigan University
Kalamazoo, MI, USA

Andrew Wojciechowski

Dept. of Computer Science
Western Michigan University
Kalamazoo, MI, USA

CS 6030 Cloud Computing — Semester Project, Milestone 1: Project Proposal

Abstract—Recorded lectures are a central course resource, but a linear video is hard to search: a student looking for one concept must skip through hours of footage. We propose the Lecture Indexer (LI), a cloud-native application that retrieves the timestamped transcripts of lecture videos from the YouTube Data API, indexes them, and lets students search a course by keyword and jump straight to the moment a term is spoken. Microsoft Azure hosts the React front end and off-site database backups; Google Cloud provides Google account sign-in and runs containerized Ingest and Search services on Google Kubernetes Engine, backed by PostgreSQL full-text search. Ingestion and search scale independently.

Index Terms—cloud-native applications, multi-cloud, Kubernetes, full-text search, YouTube Data API, educational technology.

I. INTRODUCTION

Recorded lectures let students review at their own pace, catch up on missed classes, and prepare for exams, but their value is limited by *findability*. A student who wants to revisit “lambda calculus” must guess which of dozens of hour-long recordings covers it and then skip through it. Many lecture videos already carry captions, uploaded by the instructor or generated by YouTube [1], yet YouTube offers no course-scoped search over them that returns every lecture and timestamp where a term is spoken.

LI closes this gap with two capabilities. *Ingest*: a professor registers a lecture, and LI retrieves its timed caption track and indexes it. *Search*: a student enters a keyword or phrase and receives the matching lectures, each with a transcript excerpt and a deep link to that moment. If “lambda calculus” is spoken at 03:22 of “Fall Lecture 4,” the result opens that video at $t = 202$ s.

II. PROBLEM DEFINITION AND SCOPE

A. Users and Workflow

LI serves two roles, professors and students:

1. A professor publishes a recorded class to YouTube.
2. The professor signs in with Google and registers the video.
3. LI fetches the video’s captions and indexes them.
4. A student signs in and searches the course for a keyword.
5. LI returns each matching lecture and timestamp as a link.

B. Scope Decisions

No video hosting. Storing and streaming video dominates the cost of a lecture platform, so YouTube remains the system of record for video and LI stores only text and metadata.

No transcription. LI does not run speech recognition. It retrieves existing timed captions through the YouTube Data

API [2], which removes the most expensive computation from the pipeline. A video must therefore have a caption track to be indexed. Each lecture is fetched once and served from LI’s index, so searches never consume API quota.

Google accounts. Downloading a video’s captions through the API requires authorization from the video’s owner, so professors must sign in with the Google account that owns their channel. Using Google sign-in for all users keeps one identity provider.

III. CLOUD-NATIVE JUSTIFICATION

Demand is bursty and two-sided: uploads cluster after class meetings, searches peak on weekday evenings and before exams, and both fall to near zero between terms. A monolithic server sized for peak load would idle most of the year. LI instead splits ingestion and search into separate services that scale out under load and back in when idle. Without video storage or transcription compute, steady-state cost stays near that of a small cluster and database.

IV. PROPOSED ARCHITECTURE

Presentation. Fig. 1 shows the architecture and Table I the selected services. A React single-page application is served from Azure Static Web Apps [3]. Users sign in with their Google accounts through OAuth 2.0 and OpenID Connect [4]. The resulting ID token identifies the user and accompanies every API call. When a professor registers a lecture, the sign-in also grants read access to that professor’s YouTube captions.

Application tier. Two Docker-containerized services run on Google Kubernetes Engine (GKE) behind Cloud Load Balancing. The *Ingest API* verifies that the professor owns the video, downloads its timed captions from the YouTube Data API, splits them into timestamped segments, and stores them. The *Search API* answers student queries. Each service scales independently with the Horizontal Pod Autoscaler [5], and the GKE cluster autoscaler adds or removes nodes to match.

Data tier. Cloud SQL for PostgreSQL stores courses, lectures, and transcript segments. PostgreSQL’s built-in full-text search [6] provides stemming, ranking, phrase matching, and highlighted excerpts over an inverted index, so no separate search cluster is needed. Each segment keeps its start time, so every hit maps directly to a deep link.

Backups. A nightly Kubernetes CronJob writes an encrypted `pg_dump` of the database to Azure Blob Storage [8], keeping an off-site copy with a second provider.

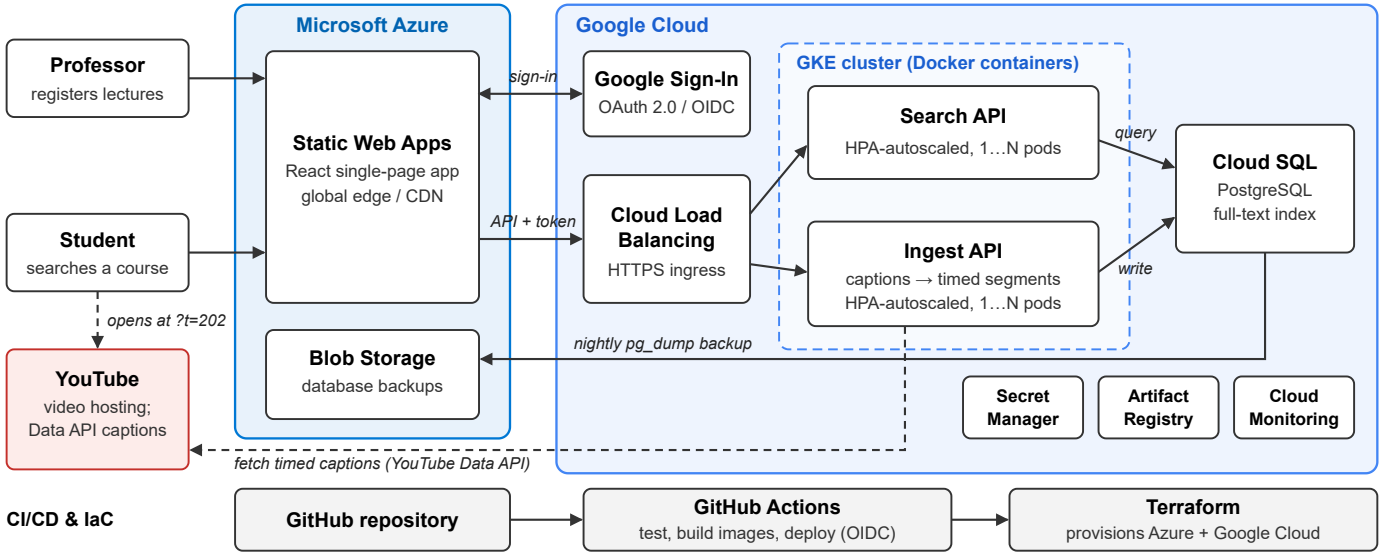


Fig. 1. Proposed architecture: the front end and database backups on Microsoft Azure; Google sign-in, containerized Ingest and Search services, and the database on Google Cloud; videos and captions on YouTube. GitHub Actions and Terraform deploy and provision both clouds.

V. TECHNOLOGY SERVICES

TABLE I
SELECTED TECHNOLOGY SERVICES

Layer	Service	Justification
Front end	Azure Static Web Apps	Edge CDN, free tier, PR previews
Backups	Azure Blob Storage	Off-site, immutable, cool tier
Identity	Google Sign-In (OAuth)	Required for caption access
Compute	GKE + Docker	Per-service pod autoscaling
Database	Cloud SQL (Postgres)	Relational + full-text search
Transcripts	YouTube Data API	Existing timed captions
Secrets	Secret Manager	Audited, via Workload Identity
Registry	Artifact Registry	Private images near cluster
CI/CD	GitHub Actions	Required; keyless OIDC deploys
IaC	Terraform	Required; Azure + GCP providers
Monitoring	Cloud Monitoring	Dashboard + alerts

VI. CLOUD PROVIDER SELECTION RATIONALE

Google Cloud for identity, the application tier, and the data tier. (1) Ecosystem fit: the YouTube Data API and Google sign-in are configured in a Google Cloud project, so hosting the backend there keeps OAuth clients, quotas, and logs together. (2) Cost: GKE’s free-tier credit covers the management fee of one zonal or Autopilot cluster [7], whereas Amazon EKS bills every cluster-hour.

Microsoft Azure for presentation and backups. Static Web Apps gives free-tier edge hosting with built-in GitHub Actions deploys. Backups in Azure Blob Storage mean an outage or account compromise on Google Cloud cannot destroy both the data and its backups.

Trade-offs. Only browser API calls and one nightly backup cross clouds, so egress is small. Two billing accounts and IAM models are managed through one Terraform codebase and keyless OIDC.

VII. SECURITY AND OBSERVABILITY

Identity management. Users authenticate with Google accounts; workloads and CI/CD use Workload Identity and

OIDC federation, not long-lived keys.

Access control. The APIs verify each ID token’s signature and audience and check the user’s role in LI: professors manage only their own courses and videos, and students get read-only search. A professor’s YouTube access token is requested with read-only scope, used during ingestion, and not retained. Least-privilege service accounts, Kubernetes RBAC, and private-IP-only Cloud SQL limit lateral access.

Secrets management. OAuth secrets, database credentials, and a write-only backup credential live in Secret Manager, never in code or images. Immutable retention protects backups from tampering.

Observability. A Cloud Monitoring dashboard will track search latency and errors, ingestion failures, YouTube API quota use, backup job status, pod count, and database load, with alerts.

VIII. CONCLUSION

LI turns hours of linear lecture video into a searchable, timestamped index; a working prototype is planned for Week 10.

REFERENCES

- [1] W3C, “Web Content Accessibility Guidelines (WCAG) 2.1,” W3C Recommendation, Jun. 2018.
- [2] Google, “YouTube Data API: Captions.” [Online]. Available: <https://developers.google.com/youtube/v3/docs/captions>
- [3] Microsoft, “Azure Static Web Apps documentation.” [Online]. Available: <https://learn.microsoft.com/azure/static-web-apps/>
- [4] Google, “Using OAuth 2.0 to access Google APIs.” [Online]. Available: <https://developers.google.com/identity/protocols/oauth2>
- [5] The Kubernetes Authors, “Horizontal Pod Autoscaling.” [Online]. Available: <https://kubernetes.io/docs/concepts/workloads/autoscaling/>
- [6] PostgreSQL Global Development Group, “Full text search.” [Online]. Available: <https://www.postgresql.org/docs/>
- [7] Google Cloud, “Google Kubernetes Engine pricing.” [Online]. Available: <https://cloud.google.com/kubernetes-engine/pricing>
- [8] Microsoft, “Azure Blob Storage documentation.” [Online]. Available: <https://learn.microsoft.com/azure/storage/blobs/>

APPENDIX. AI USAGE

TABLE II
AI TOOL USAGE IN PREPARING THIS PROPOSAL

Item	Details
AI tool used	Claude
Purpose	1. For initial brainstorm of project ideas 2. To formalize our proposal into IEEE format and create PDF
Example prompt	<i>“Read through the requirements and the proposed application, create an IEEE proposal from those outlines.”</i>
Generated output used	The file <i>Project Proposal - IEEE.pdf</i>
Verification performed	We did iterative revision until the proposal matched our idea and problem.